

Assignment # 1 - Exploring Requirement Engineering Tools

Group# 19

Members:

- 1) BSSE23113: Muhammad Samer Nisar
- 2) BSSE23043: M. Umar Younas

Name: M. Umar Younas

Tool: ScrumWise

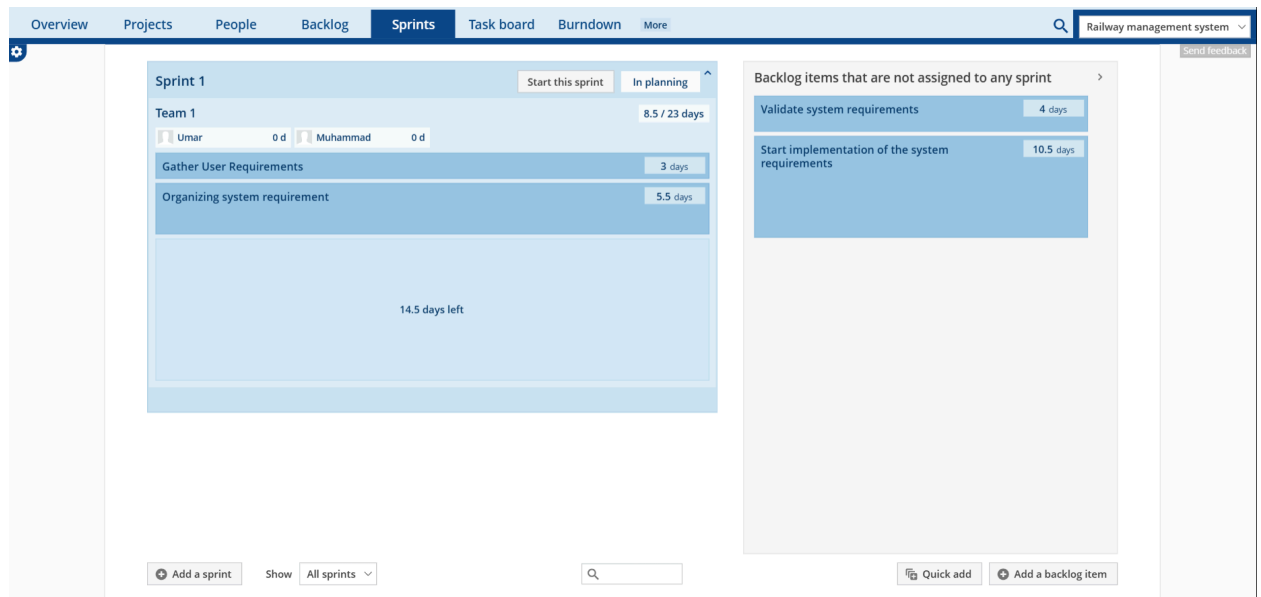
Owner: Esben Krag Hensen

Purpose: ScrumWise is a Software Engineering platform that helps developers work in an agile and scrum environment. It helps the teams plan, track, and manage their work in an agile environment, focusing on iterative development and continuous improvement through real-time team collaboration. ScrumWise is one of the most user-friendly software for requirements gathering and fits best for beginner software engineers as it not only has a simple menu but also offers a visual representation of all the work.

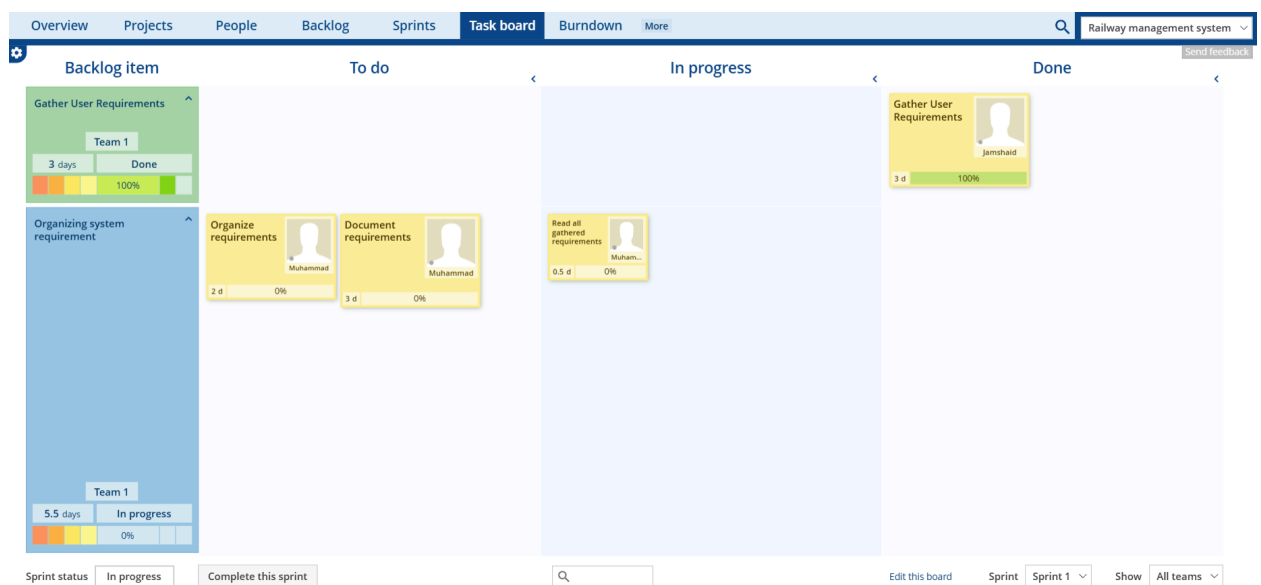
Primary Features:

- **Sprint Planning and Management:** Facilitates planning sprints, tracking the progress of each spring, and managing backlogs where developers can see all the sprints along with

their current progress.



- **Task Boards:** Provides visual Kanban/Scrum boards to manage tasks, user stories, and the status of these tasks and user stories.



- **Burndown Charts:** Tracks the progress of each sprint's work done over the time.



- **User Story Management:** Allows for the creation, detailing, tracking, and prioritization of user stories and tasks.

Backlog items that are not assigned to any sprint >

As a user I want maximum security of user data Ready for estimation

As a user I want to deal with a user friendly... Ready for estimation

As a user I expect 24/7 uptime of the system Ready for estimation

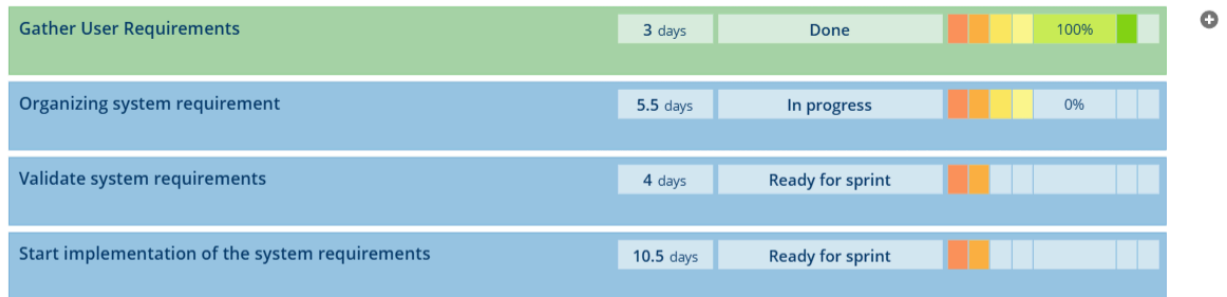
Validate system requirements 4 days

Start implementation of the system requirements 10.5 days

 Quick add

 Add a backlog item

- **Reporting and Analytics:** Offers various reports to analyze team performance, sprint progress, and workload.



- **Tools to collaborate:** ScrumWise Includes real time communication features, such as messages and notifications and announcements.

Messages
Backlog
Sprints
Task board
Burndown
More

Congrats on successfully gathering system requirements!
📢 Announcement by Umar · Less than 1 minute ago
Kudos to Jamshaid for completion of the first sprint!

Welcome to the team
📢 Announcement by Umar · 4 minutes ago
Hello everyone, let's get started with the sprints!

Systematic Requirement Engineering Support in ScrumWise

1. Requirement Gathering:

- **User Stories:** ScrumWise allows teams to create and manage user stories, which serve as the primary unit of requirement gathering in an agile environment. Teams can store the user requirements in the form of user stories, they can also add details to the requirements, and also prioritize one requirement over the other, and put them in the backlog. The user stories can also be assigned to different team members and ScrumWise starts to track the progress of each user story.

Item #5

X

Name

As a user I want to deal with a user friendly menu

Description

The menu should contain only the most commonly used tools and features, the menu should not be clustered with buttons. It should have an option to take a new user on a tutorial of the most common features of the software.

Type

Feature ▾

Created by

Muhammad ▾

Rough estimate

10 points

Tags

Create a new tag

Status

In progress ▾

0%

Estimate

10 days

Remaining

10 days

Sprint

Sprint 1 ▾

Team

Team 1 ▾

Assigned to

Jamshaid ▾

Delete

Show activity

Add a task

Add a comment

More ▾

Close

- **Backlog Management:** ScrumWise Provides a structured backlog where all user stories and tasks can be collected and organized(as shown in “User Story Management”). This backlog is a key component in gathering and refining requirements over time.

2. Requirement Analysis:

- **Sprint Planning:** Facilitates sprint planning sessions where requirements (user stories) are reviewed, estimated, and assigned to sprints. This helps in analyzing the requirements based on their priority and complexity.
- **Burndown Charts:** Offers burndown charts to visualize progress towards completing user stories and tasks, helping teams analyze how effectively requirements are being addressed and identify any issues. (as shown in the “Burndown Charts”)

3. Documentation:

- **User Story Details:** Allows detailed documentation of user stories, including acceptance criteria, descriptions, and attachments. This documentation helps ensure that all aspects of the requirements are captured and understood.
- **Task Comments:** Provides a platform for adding comments and notes to tasks and user stories, allowing for further clarification and discussion about the requirements.

4. Requirement Management:

- **Task Boards (Kanban/Scrum):** Manages the lifecycle of requirements through visual task boards, enabling teams to track the status of each user story and task as they move through various stages of completion. Each user's requirements can either be in
 - 1. To Do
 - 2. In progress
 - 3. Done

Through these three divisions, the processing of requirements is managed.

- **Prioritization and Refinement:** Supports the prioritization and refinement of requirements within the backlog. Teams can reorder user stories, adjust priorities, and break down larger requirements into smaller tasks.
- **Reporting and Metrics:** Offers various reports and metrics to monitor the progress and effectiveness of requirement management, such as sprint progress, workload distribution, and performance analytics.

4) Case Study Application:

Case Study: Applying ScrumWise to Develop a Library Management System for Information Technology University Lahore.

Project Overview: The project involves developing a library management system for a university with the name ITU (Information Technology University), Lahore. The system aims to automate library operations that occur physically in the library, like book management, borrowing and returning books, managing different types of accounts, etc.

Project Strategy: Agile

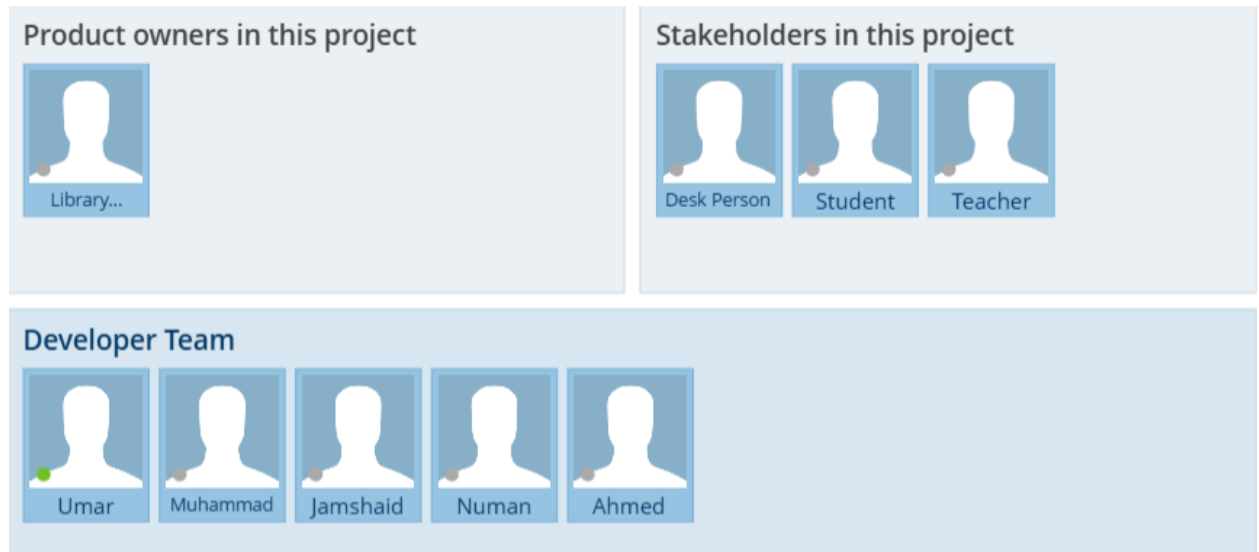
Reason For Choosing Agile Strategy: As the library management system involves continuous meetings with stakeholders, there's a high chance that the requirements will be changed in the middle of the project and the development strategy should support that. Another reason is that the library management system should support updates and corrections throughout its development and agile is the best way to go in this condition.

Tool Selection: ScrumWise.

Reason For Selecting ScrumWise: This tool supports agile development and so therefore fits the best for this project.

Case Study Steps:

1. **Establishing developer team and stakeholder list:** The first step of the development is to create a team and also create a record of the people who are the stakeholders and also keep the record of the product owner. All the respective people will be acknowledged in the ScrumWise menu.



1. Requirement Gathering:

- **Initial Setup:** The project begins with gathering requirements from stakeholders of Information Technology University, including librarians, students, and faculty because these are the actors of our software. In ScrumWise, these user stories are put in the backlog. The common stories will be,
 - i. "Manage librarians' accounts"
 - ii. "Add librarian's accounts"
 - iii. "Remove librarians' accounts"
 - iv. "Add student accounts"
 - v. "Remove student accounts"
 - vi. "Set limited permissions for the student accounts"
 - vii. "Add books"
 - viii. "Add book details"
 - ix. "Remove books"
 - x. "Record lending status of books"
 - xi. "Keep track of the duration for each lent book"
 - xii. "Notify the librarian about lending books"
 - xiii. "Notify the respective librarian when the book returning deadline surpasses"
 - xiv. "Notify the librarian if student has returned a book"
 - xv. "Keep track of books present in library"
 - xvi. "After student returns the book, verify if the condition is acceptable"

- xvii. “Manage each student’s record if he/she owns a book to the library or not”

All of these user stories will be created inside the ScrumWise menu and each of these stories will be detailed, and then assigned to different team members for completion.

- **Backlog Creation:** All collected requirements are entered into ScrumWise, where they are organized and prioritized. For example, “Manage Librarian’s Accounts” might be given higher priority than “Add Student Accounts”

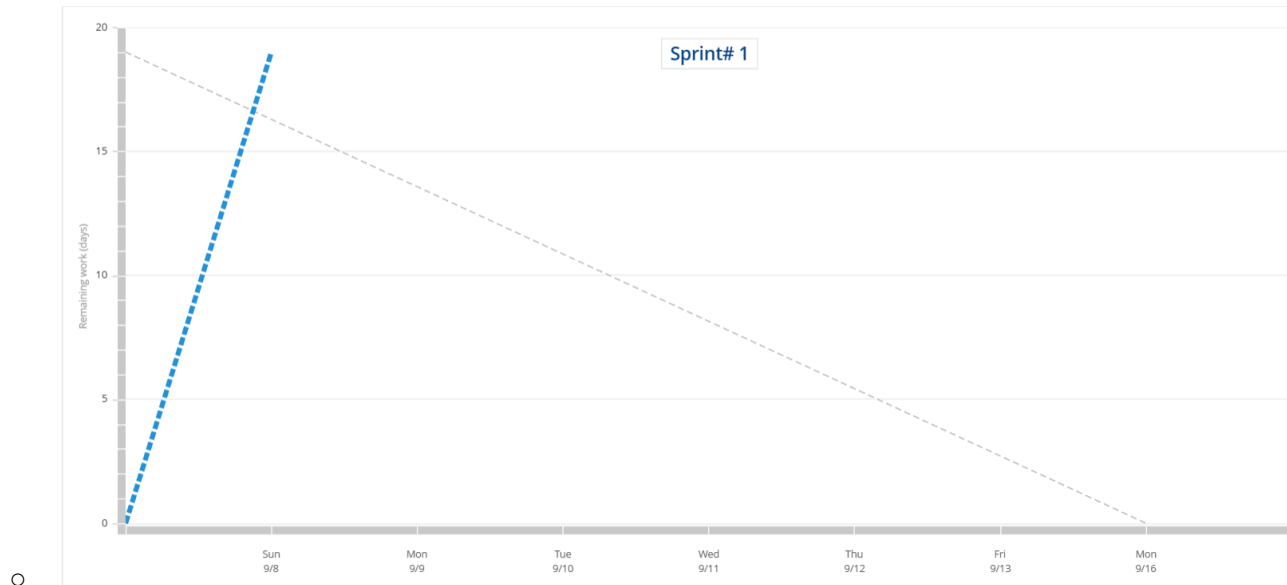
Backlog items that are not assigned to any sprint		>
Manage Librarians Accounts	1 day	
Add Student Accounts	1 day	
Add Librarians' Accounts	1 day	
Remove Librarians' Accounts	1 day	
Remove Student Accounts	4 days	
Set Limited Permissions For The Student Accounts	1 day	
Add Books	1 day	
Add Book Details	2 days	
Remove Books	5 days	
Keep Record Of Lending Status Of Books	2 days	
Keep Track Of Time For Each Lent Book	2 days	
Keep Track Of Which Librarian Account Lent Books To...	2 days	
Notify Respective Librarian If The Lent Book Was...	3 days	
Let Librarian Update Book Availability Status	2 days	

2. Requirement Analysis:

- **Sprint Planning:** During the planning of sprint, the team manager reviews the requirements present in the backlog. He estimates the effort required for each story, selects the highest-priority items, and assigns them to the upcoming sprint. For instance, the team might plan to implement “Remove Student Accounts” and “Add Librarians’ Accounts” in the first sprint.



- **Burndown Charts:** As the work goes on. ScrumWise tracks the performance in burndown charts that show the amount of work remaining. This helps the team analyze their progress and make adjustments if necessary.



3. Documentation:

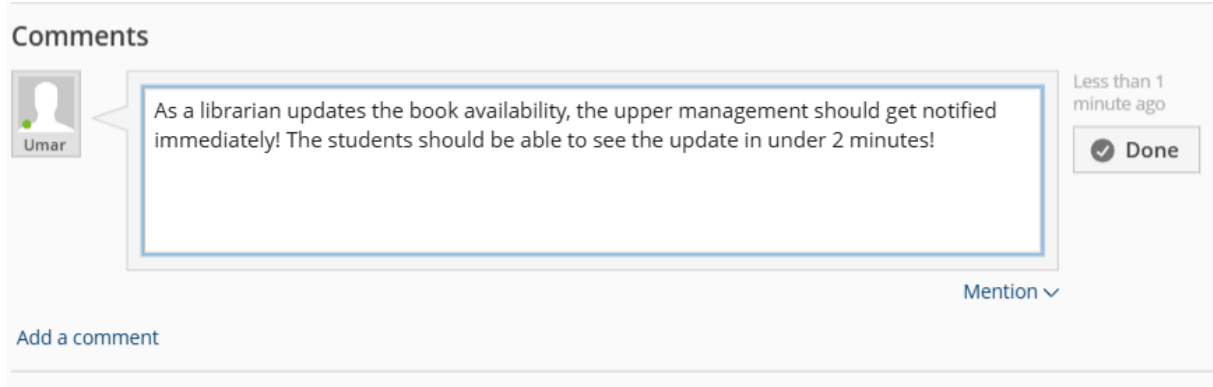
- **User Story Details:** Each user story in ScrumWise includes detailed documentation, such as acceptance criteria and any specific requirements. For instance, the user story “Add Book Details” might include details like mandatory fields (title, author, ISBN) and the process for updating existing records.

The screenshot shows a 'Backlog item' form for 'Add Book Details'. The form has the following fields:

- Name:** Add Book Details
- Description:** The following details should be added.
1) IBN Number
2) Author Name
3) Release Date
- Type:** Feature (dropdown menu)
- Created by:** You (dropdown menu)

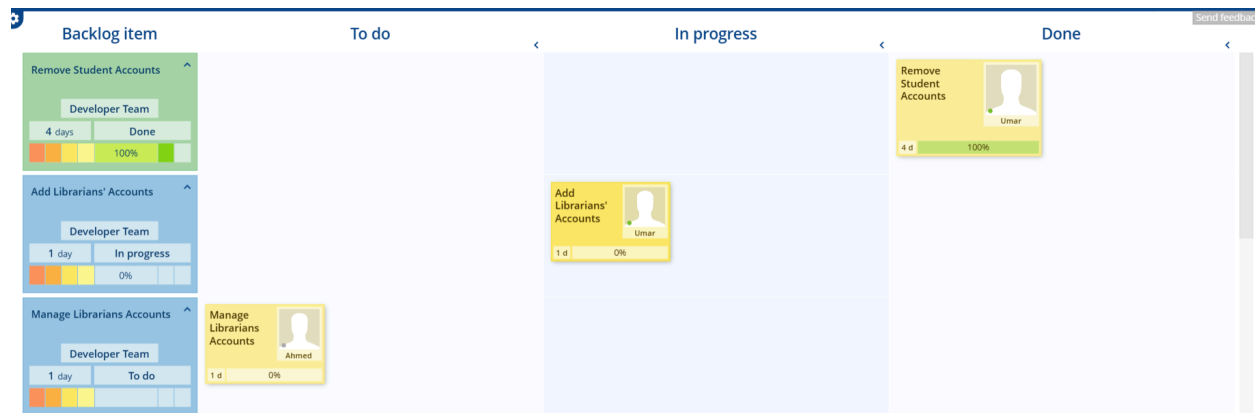
The Description field is highlighted with a blue border.

- **Task Comments:** Team members can add comments and notes to user stories and tasks to clarify requirements or discuss implementation details. This ensures that all team members don't face any problems in collaborating.



4. Requirement Management:

- **Task Boards:** ScrumWise provides visual task boards (Kanban or Scrum) where tasks are moved through stages such as “To Do,” “In Progress,” and “Done.” This helps track the status of each requirement as it progresses through the development process. For example, once the “Remove Student Accounts” feature is implemented, it moves from “In Progress” to “Done.”, All of the other features remain in “To Do” if their development has not yet started.



- **Prioritization and Refinement:** The backlog is continuously refined and reprioritized based on feedback and changing requirements. For instance, if there's a change in the requirement that requires the need for more details in “Add Book Details”, this requirement can be added backlog and can also be prioritized.

5. Team Collaboration:

All the team members will be able to do real-time collaboration, as each team member will be able to talk to each other through messages and the team members will be able to make announcements to let everyone know about their progress and if they're facing any problems. This allows everyone to collaborate like they're all present inside a single room, maximizing efficiency.



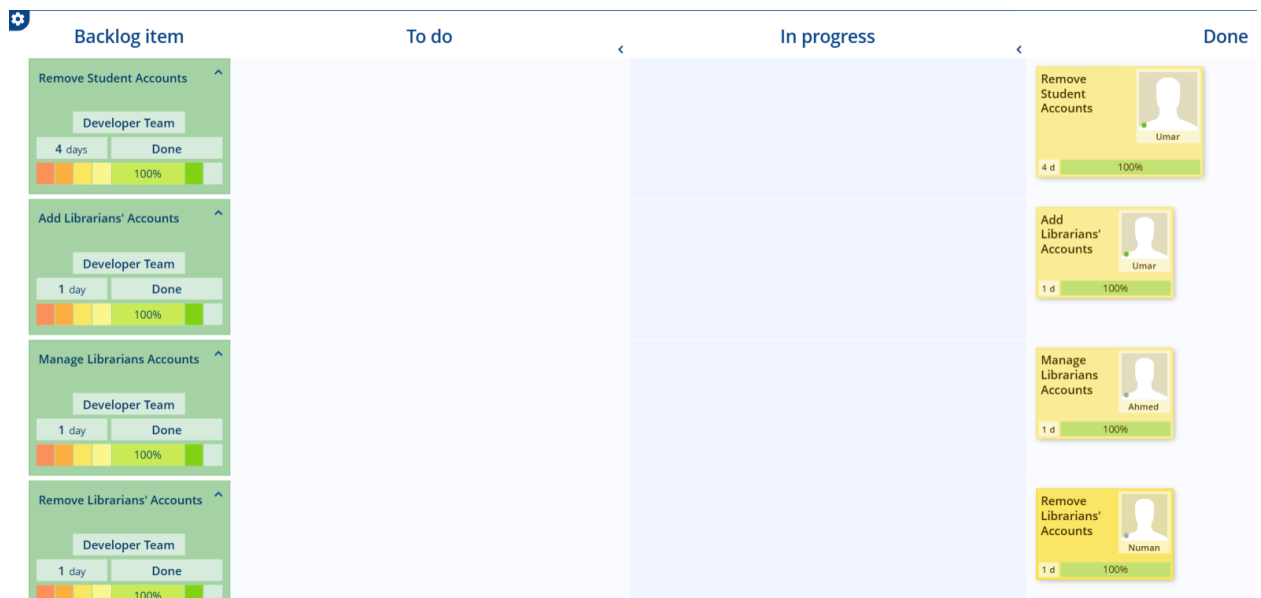
How's the progress going?
🗨 Question by Umar · Less than 1 minute ago
Hello Everyone, I hope you're all enjoying working! Please let me know about your current progress! Cheers!



Feel Free To Discuss Any Problems!
📢 Announcement by Umar · Less than 1 minute ago
If someone is having any sort of issue, remember that we're all here to discuss that with each other, so feel free to share any ambiguities with me and your team mates!

6. End Of Development:

Once all the user requirements have been sent to “Done” in the Task Board, it means that the respective sprint has been completed and the current user requirements have been implemented! Marking the end of that development phase until new requirements are gathered.



How ScrumWise Supports Systematic Requirements Engineering

1. Requirement Gathering:

- **Initial Setup:** ScrumWise facilitates the initial setup by allowing the creation of user stories directly in its backlog
- **Backlog Creation:** Once gathered, requirements are organized and prioritized in the ScrumWise backlog. High-priority stories are placed higher in backlog.

2. Requirement Analysis:

- **Sprint Planning:** ScrumWise supports sprint planning by enabling the team to review and estimate user stories. The team can select high-priority stories for the sprint and plan their development. This ensures that the most important requirements are addressed in a structured manner.
- **Burndown Charts:** ScrumWise provides burndown charts that track the progress of work remaining versus the time left in the sprint. These charts help the team analyze their progress and make adjustments as needed, ensuring that the development process stays on track and that requirements are met within the sprint timeline.

3. Documentation:

- **User Story Details:** Each user story in ScrumWise can be detailed with specific requirements. Ensuring that all requirements are documented and understood by the team.
- **Task Comments:** Team members can add comments and notes to user stories and tasks to discuss and clarify requirements. This feature ensures that all team members have a shared understanding of the requirements and that any changes or updates are communicated effectively.

4. Requirement Management:

- **Task Boards:** ScrumWise provides visual task boards (Kanban or Scrum) where tasks are moved through stages like "To Do," "In Progress," and "Done." This visual representation helps track the status of each requirement, ensuring that they are managed efficiently throughout the development process.
- **Prioritization and Refinement:** The backlog is continuously refined and reprioritized based on feedback and evolving requirements. For example, if more details are needed

in “Add Book Details,” the requirement can be updated and reprioritized, ensuring that the backlog remains relevant and aligned with project needs.

5. Tracking Performance:

Through the feature of the “BurnDown” Graph, the performance of the whole team is kept in view, helping in further planning.

6. Team Collaboration:

- **Real-Time Communication:** ScrumWise facilitates real-time collaboration among team members through messaging and announcements. This feature helps team members stay connected, share progress, and address any issues promptly, enhancing overall team efficiency and coherence.

7. End of Development:

- **Completion Tracking:** At the end of each sprint, when all user requirements are marked as “Done” on the task board, it signifies the completion of that development phase. This systematic tracking ensures that each phase of development is completed and reviewed before moving on to new requirements and starting of a new sprint.

Critical Analysis of ScrumWise

1. Limitations and Challenges:

- **Limited Documentation Capabilities:**
 - **Challenge:** ScrumWise doesn't offer detailed documentation features compared to other requirements management tools focused on documentation. This can be a limitation when a lot of documentation is required.
 - **Mitigation:** ScrumWise should be used alongside other documentation tools in case detailed documentation is required.
- **Complexity in Managing Large Backlogs:**
 - **Challenge:** Managing and prioritizing a large number of user stories and tasks can become messy because all the work is manual and there is no automated organization of user stories.
 - **Mitigation:** Regularly refine the user stories manually.
- **Integration with Other Tools:**
 - **Challenge:** The integration capabilities of ScrumWise with other tools and systems might be limited compared to other platforms.
 - **Mitigation:** All the team members should try to use ScrumWise so that no integration problem occurs.

- **Real-Time Collaboration Limitations:**
 - **Challenge:** ScrumWise's real-time collaboration features may be less advanced as it doesn't offer voice-to-voice communication, it just relies on messages and notifications.
 - **Mitigation:** Use ScrumWise with dedicated communication and collaboration tools, such as Microsoft Teams or Google Meet, to enhance collaboration.
- **Lack of Diagram Support:**
 - **Challenge:** ScrumWise does not support diagramming, which can be a significant limitation for projects that require visual representations of processes, workflows, or system designs.
 - **Mitigation:** Use complementary tools for diagramming, such as Lucidchart, or draw.io, to create and share visual representations. Integrate these diagrams with ScrumWise by including links or attachments within user stories or tasks.

Name: Muhammad Samer Nisar

Tool: Pivotal Tracker

Owner: VMware, Inc

Overview of Pivotal Tracker:-

Primary Goal: The primary goal of Pivotal Tracker is to provide them with a flexible platform that can manage requirements for the agile product perfectly. It also helps them to calculate the difficulty of a specific requirement. It also helps in setting the number of people working in one iteration and on a specific task.

Primary Features:

1. Requirements Gathering:

- **Stories Creation:** We can write our requirements in the form of stories. A story can be of 4 types i.e. Feature, Bug, Chore, Release. Feature represents the requirement proposed by the user/stakeholder. The bug represents a fault that has been detected in our system and needs to be fixed. Chore represents the requirements proposed by the developers not by end users or stakeholders. The release represents the final release of the iteration or a product. We can then assign people to the story. After creating a story we can place it in its appropriate position.
- **Labels:** We can assign multiple labels to each story and in this, we can group our requirements that are associated with each other in some way.
- **Comments on Stories:** Each story has its own comment section where other developers can write a comment suggesting something about the story or pointing out any ambiguity or bug.
- **Tasks in Stories:** We can set multiple tasks in each of our stories to help story participants implement the story.
- **Setting the Difficulty Level:** The difficulty level varies from 1-8 and it depends on the ability of the team. We can set the difficulty level for each story after a proper talk with project co-workers and can have a rough estimate of the time that will be taken by that story.
- **Overview of All the Iterations:** Pivotal Tracker provides the general overview of all the iterations and iterations include multiple stories.

2. Analysis

- **Set Priority:** We can prioritize the requirements and place them on top of each other(in the form of stories) to maintain the order of the development process.
- **Ice Box:** We can move unnecessary stories or stories that are optional to implement and can be implemented anytime during the process in the ice box so that they won't bother with the process of development.
- **Epics:** We can create epics to group a large number of interrelated requirements by creating epics.

3. Documentation

- **Description in Stories:** While creating stories we have an option of adding description in that box we can thoroughly describe the functionality of that story and it is what makes the documentation easy.
- **Done Section:** In this section all the completed stories are shown so that we can continue to add features to our documents that have been completed.

4. Management

- **Current/Backlog:** In this section, we view all our ongoing and upcoming iterations. And stories placed in Ice Box.
- **Blocked Stories:** We can use the block method to temporarily prevent developers from starting a story until they have not completed the previous stories in this way we can manage the workflow.
- **Review:** There are 4 types of reviews i.e. code, security, test(QA), and design. We can review the stories and mark those reviews in 4 options i.e. unstated, pass, reviser, in review..

Case Study Application:

Project: E-commerce Platform

Requirements:-

- Implementation of User class for all types of users on the platform
- Implementation of admin functionality i.e. admin can monitor products and remove the one that goes against the terms and conditions.
- Implementation of buyer functionality i.e. buyer can set preferred categories and buy the items on the feed.
- Implementation of seller functionality i.e. seller can publish his products on the platform.

in progress x general functionality x

CODE

Paste link to pull request or branch...

TASKS (0/2)

☐ Create a class named user



☐ create a the member variables and member functions

+ Add a task

ACTIVITY

Sort by Oldest to newest



@samern



Implement it correctly

Like... · Sep 8, 8:10 pm

Write

Preview

Formatting help



Add a comment or paste an image



Post comment

Current Iteration/Bac... 10 + Add Story : X

0 of 2 points 1 • 2 - 8 Sep • 100%

★ = User class Implementation (SN) Finish ☐

general functionality, in progress

Admin Functionality 

 ID    Cancel Save

STORY TYPE ★ Feature ▼

POINTS 3 Points ▼

REQUESTER SN Samer Nisar ▼

OWNERS SN Samer Nisar +

FOLLOW THIS STORY (1 follower) ☒

BLOCKERS

 complete this first (#188237729 ☐)



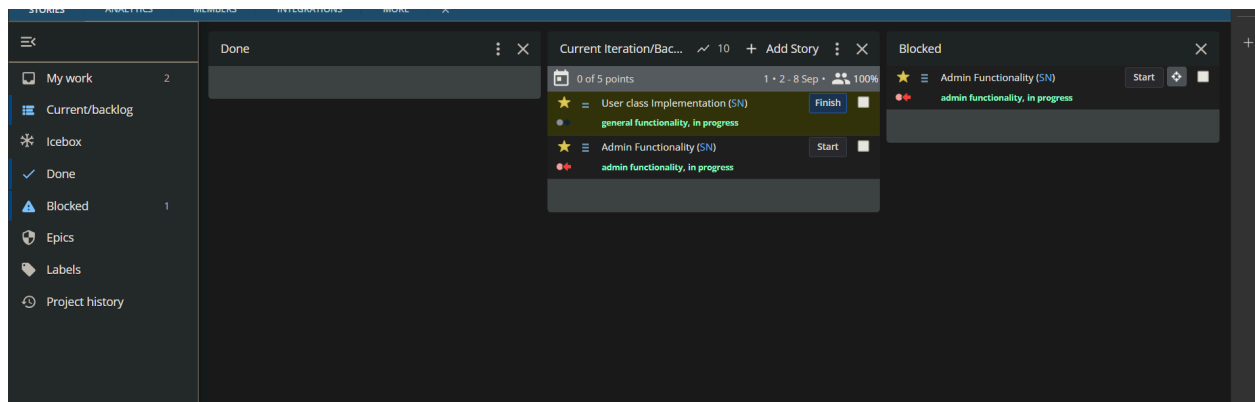
Cancel Add

DESCRIPTION

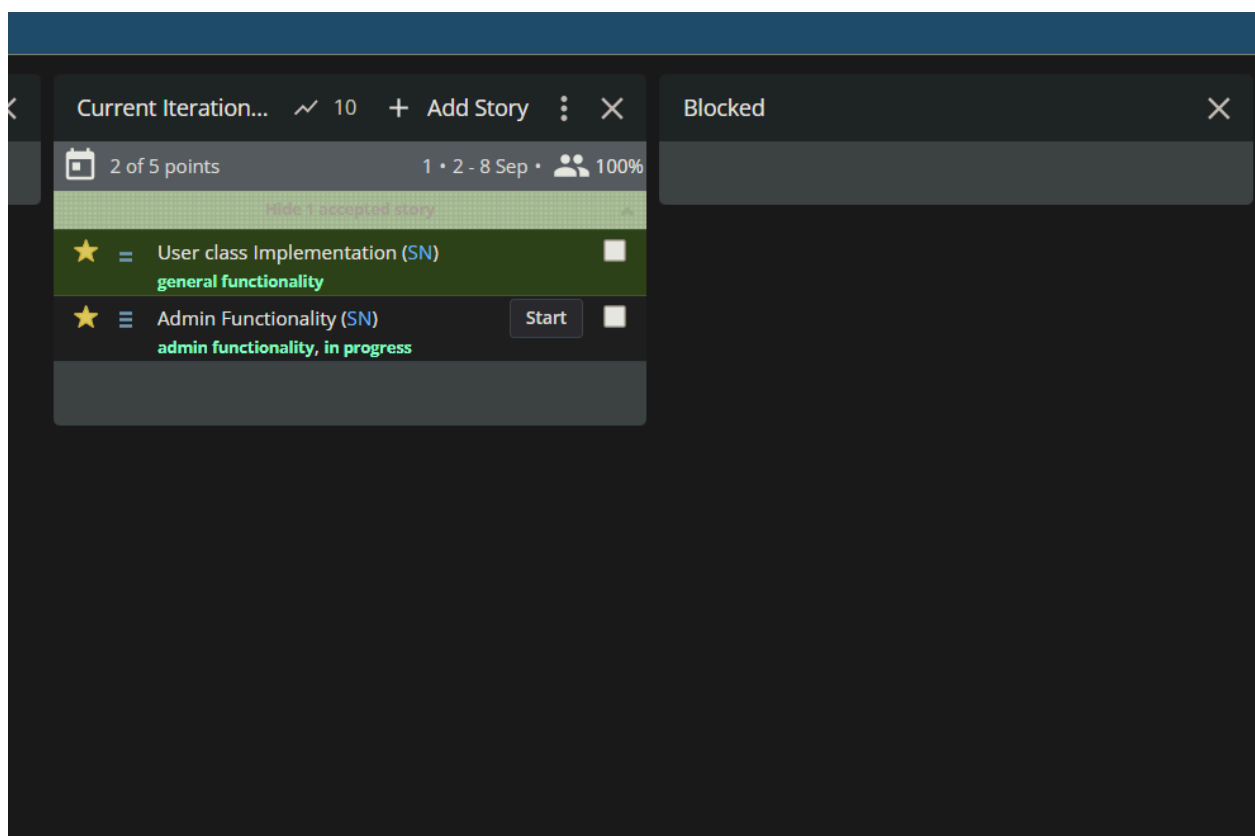
Write Preview [Formatting help](#)

Implementation of admin functionality i.e admin can monitor products and remove the one that goes against the terms and conditions.

In this picture, a blocker has been placed that will prevent this story's progress until the user class is not implemented.



This is an overall overview of the situation right now.



Now that the user class has been implemented we can implement the admin functionality.

0 of 0 points 1 • 2 - 8 Sep • 100%

User class Implementation

ID     Cancel Save

STORY TYPE ★ Feature ▼

POINTS ■ 2 Points ▼

REQUESTER SN Samer Nisar ▼

OWNERS SN Samer Nisar +

FOLLOW THIS STORY (1 follower) ☒

BLOCKERS

+ Add blocker or impediment

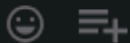
DESCRIPTION

Write

Preview

[Formatting help](#)

This class contains general functionality for all type of users.



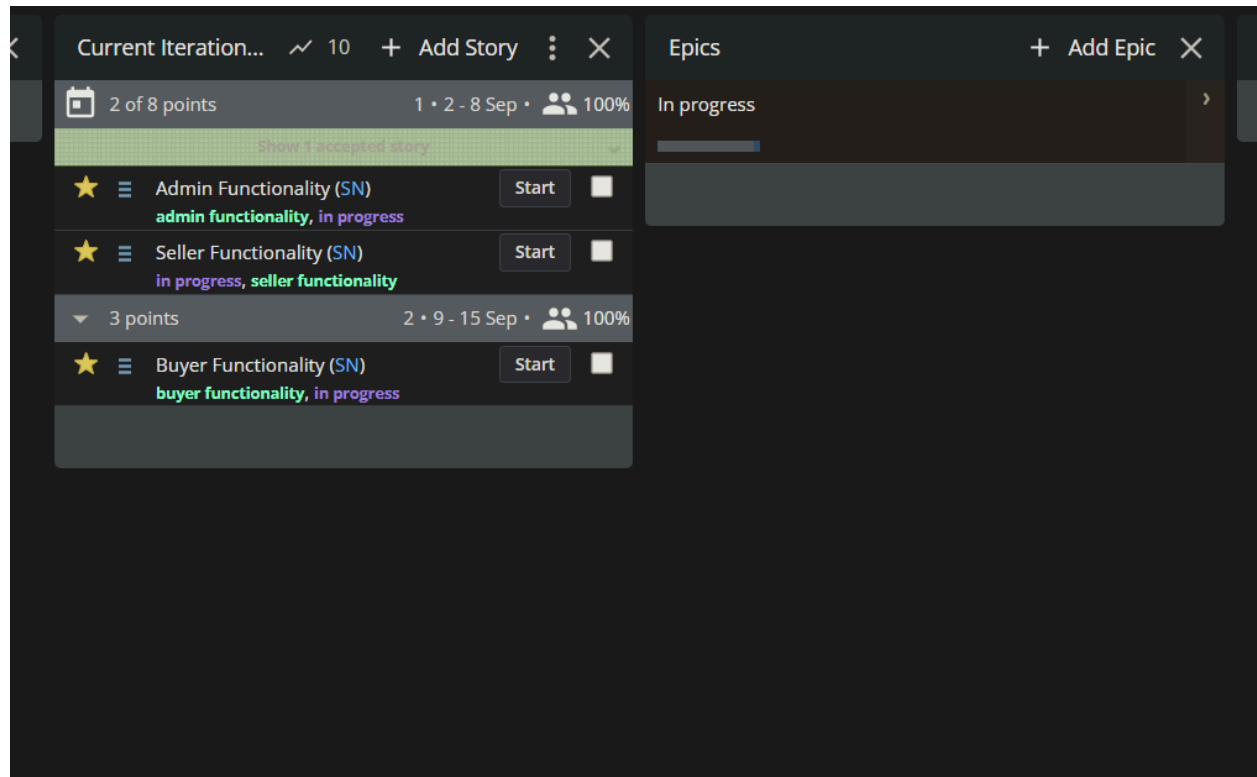
Cancel

Add description

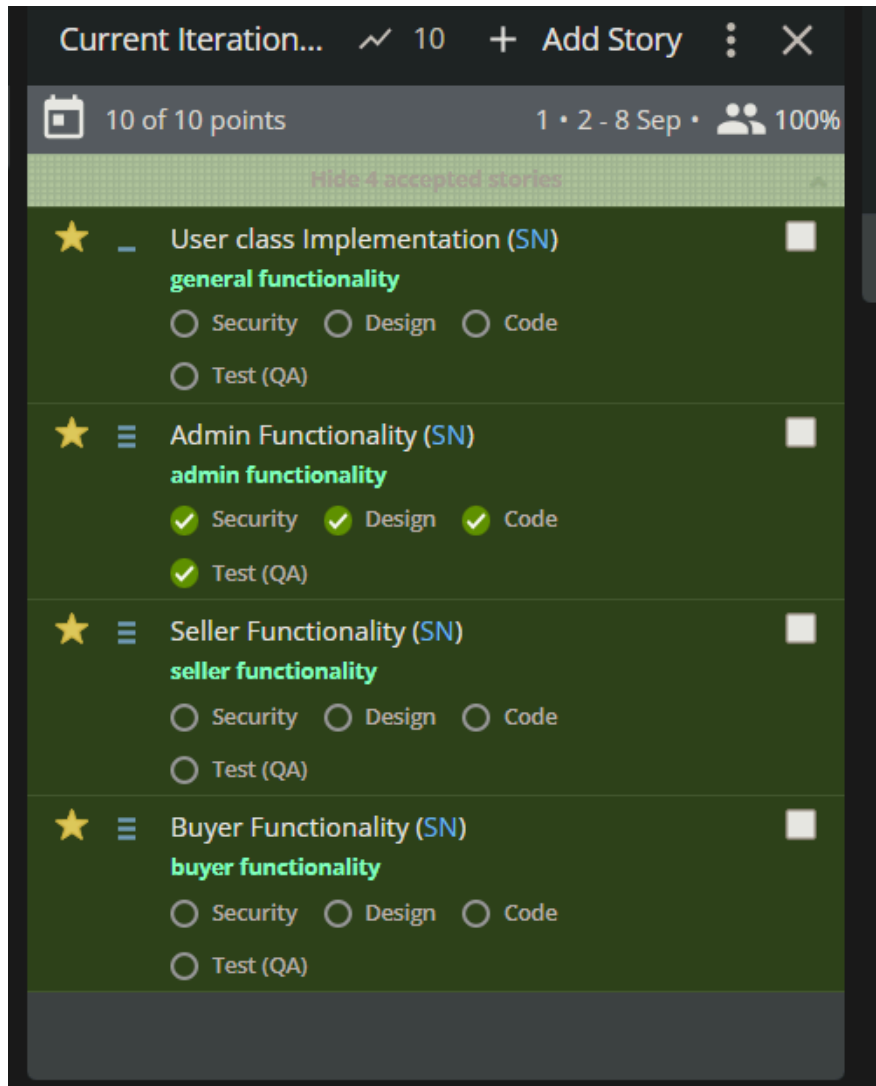
LABELS

in progress ×

general functionality ×



I have created an in-progress epic that groups all under development requirements.



Here I have applied 4 types of reviews in each Story.

Critical Analysis

1. Limited Requirement Management:

- **Problem:** It lacks some detailed functions such as requirement traceability and requirement version management.
- **Mitigation:** It can be solved using a Pivotal tracker and other software such as (Doors Next (jazz)).

2. Limited Handling of Requirement Changes:

- **Problem:** It lacks the traceability of the requirements that have been changed.
- **Mitigation:** Implementing change management processes and integrating with version control tools.

3. Gaps in Integration:

- **Problem:** It has limited integration with requirement engineering tools.
- **Mitigation:** Use APIs or custom integration to connect the pivotal tracker with other tools.

4. Workflow:

- **Problem:** Rigid agile workflow is not suitable for all types of products.
- **Mitigation:** Use automation scripts or plugins to enhance the workflow.

5. Scalability Issues:

- **Problem:** Large backlogs will probably become unmanageable.
- **Mitigation:** Segment backlogs, use tags, and prioritize tasks effectively.

3. Comparison

Tool Name	Systematic Process Support	Features	Collaboration	Integration	Cost
ScrumWise	It supports a structured process very well due to its Kanban Board approach, But it lacks the support for diagrams, making it hard to structure user cases in graphical form.	1. Scrum Approach 2. Kanban Board 3. Real Time Communication	It enables an effective collaboration through the feature of real-time communication via messages and announcements	It does not support integration with other platforms	9\$/Month 90\$/Year
Pivotal Tracker	It supports a structured process using an iterative	1. User Stories 2. Iterations(Sprint	It lets users collaborate with team members in	It does provide integration with other platforms.	Free Plan: 0\$ Startup Plan:10\$/m

	approach within the agile methodology. It also lacks the diagram functionality.	s) 3. Epics 4. Labeling and Tags	real time, with instant updates and notifications on story status, comments, and progress.		Standard Plan: 6.5\$/m Enterprise Plan: Custom pricing (based on specific needs)
--	---	--	--	--	--
